예측시장 데이터 파이프라인 학습서

Contents

1 예측시장 데이터 파이프라인이 푸는 문제 **1**

2 왜 Kafka 인가, 왜 Redis 와 ClickHouse 둘 다인가 **3**

3 스키마를 계약으로 두는 설계 **5**

4 델타에서 스냅샷을 재구성한다는 것 **7**

5 at-least-once 에서 exactly-once 로 **9**
5.1 배경: " 적어도 한 번" 과" 정확히 한 번" 이라는 말이 왜 미끄러운가 9
5.2 앱 내부 큐: at-least-once 조차 아닌 구간 10
5.3 브로커부터 ClickHouse 까지: at-least-once 가 성립하는 구간 . . . 11
5.4 파이프라인 전체 지도 12
5.5 DB 단에서 exactly-once 로 접는 방법 12
5.6 왜 Kafka 트랜잭션 (EOS) 이 아니라 DB 중복 제거인가 14
5.7 정직하게 짚는 잔존 위험 14
5.8 한 줄로 보는 경로별 요약 15

6 예측과 트레이딩 **17**

Chapter 1

예측시장 데이터 파이프라인이 푸는 문제

이 장은 Kalshi 와 Polymarket 같은 예측시장 (prediction market), 그리고 Binance·Coinbase 같은 현물 거래소 (spot exchange) 에서 BTC 관련 데이터를 실시간으로 모으는 일이 왜 단순한"API 호출 반복" 이 아니라 별도의 설계를 요구하는 문제인지를 다룬다. 피드가 끊기고, 메시지가 중복되고, 순서가 뒤집히는 조건에서도 하나의 오더북 (orderbook) 상태를 일관되게 유지해야 한다는 요구가 이후 모든 장 (장 2 부터 장 5 까지) 의 설계 결정을 낳는다.

[**TODO**] §1–§2 의 목표 나열과 §3.3 BTC 한정 근거를 바탕으로, " 왜 이 파이프라인이 필요한가" 를 독자가 처음 만나는 문제로 풀어 쓴다. book.md 원칙 8(문제 → 처방) 을 따라 이 장 전체가 2 장 이후의 각 결정을 예고하는 방식으로 작성.

Chapter 2

왜 Kafka 인가, 왜 Redis 와 ClickHouse 둘 다인가

이 장은 세 개의 producer 가 원격 Kafka 로 모이고, 거기서 다시 Redis(현재 상태)와 ClickHouse(이력) 로 갈라지는 토폴로지가 왜 그 모양인지를 다룬다. 핵심 질문은 하나다—" 현재 상태" 와" 전체 이력" 은 왜 같은 저장소에 담을 수 없는가. 이 질문의 답이 장 4 의 오더북 재구성 설계와 장 5 의 전달 보장 설계를 함께 떠받친다.

[**TODO**] §2 비즈니스 흐름, §4.3(Redis vs ClickHouse 표), §10 로컬 실행 전제를 바탕으로 Kafka 를 중앙 버스로 둔 이유와 Redis/ClickHouse 역할 분리를 설명한다.

Chapter 3

스키마를 계약으로 두는 설계

이 장은 proto 메시지 → codec → ClickHouse DDL 로 이어지는 3 단계를” 같은 계약을 다른 형태로 반복하는 사슬” 로 보는 관점을 다룬다. `venue` 필드 하나를 추가하는 결정이 왜 병렬 테이블이나 별도 마이그레이션 없이 끝나는지가 핵심이며, 이 계약의 견고함이 장 5 에서 다루는 정렬 키 (sort key) 기반 중복 제거의 전제가 된다.

[**TODO**] §3(스키마 3 단계, venue 차원 도입 근거, 정렬 키에 venue 를 선두에 두는 이유) 를 바탕으로” 계약이 하나여야 하는 이유” 를 설명한다.

Chapter 4

델타에서 스냅샷을 재구성한다는 것

이 장은 거래소가 매번 전체 호가창을 보내지 않고 변경분 (delta) 만 보내는 이유, 그리고 그 변경분을 누적해 전체 스냅샷 (snapshot) 을 다시 만들어내는 재구성기 (reconstructor) 가 왜 WS 연결을 직접 쥔 producer 안에 있어야만 하는지를 다룬다. 이 장에서 정리하는 `BookEvent`/`VenueOrderbookAdapter` 경계는 장 5 에서 다루는 순서 보장 문제 (§4.4 의 last-write-wins 가드) 의 전제가 된다.

[**TODO**] §4.0–§4.6 을 바탕으로 재구성이 producer 에 남아야 하는 이유 (seq 갭 감지 시 reconnect 권한), Redis 라이터가 무상태인 이유 (A 안 채택 근거), venue 중립 어댑터 경계를 설명한다.

Chapter 5

at-least-once 에서 exactly-once 로

In brief.

Why 스트리밍 파이프라인은" 죽었다 다시 살아나는" 일이 늘 일어난다. 그때마다 메시지가 사라지거나 (유실) 두 번 들어오는 (중복) 일이 생기는데, 이 파이프라인이 감당해야 하는 진짜 문제는 유실을 완전히 없애는 것이 아니라 어디서 유실이 나는지를 정확히 알고, 그 나머지 구간에서는 중복을 안전하게 접는 것이다.

When producer 가 거래소 WS(웹소켓, WebSocket) 연결을 받아 Kafka 로 내보내는 순간부터, ClickHouse 의 데이터 테이블에 행이 최종적으로 쌓이는 순간까지 전 구간에 적용된다.

Where 이 장의 결론은 예측시장 (Kalshi, Polymarket) 과 현물 (Binance, Coinbase) 가족 모두에 동일하게 적용된다. `venue` 축 (장 3) 이 이 구간의 보장 수준을 바꾸지 않기 때문이다.

How 앱 내부 메모리 큐 구간은 보장이 아예 없다는 사실을 받아들이고, 그 뒤 구간 (브로커부터 ClickHouse 까지) 은 정렬 키 *(sort key)* 가 완전히 같은 행만 접는다는 ClickHouse `ReplacingMergeTree` 의 규칙 하나로 중복을 흡수한다.

5.1 배경: " 적어도 한 번" 과" 정확히 한 번" 이라는 말이 왜 미끄러운가

스트리밍 파이프라인을 처음 설계할 때 누구나 듣는 말이 있다. "at-least-once(적어도 한 번 전달) 로 받고 exactly-once(정확히 한 번) 로 저장한다." 문제는 이 두 단

어가 파이프라인 전체의 성질처럼 들리지만, 실제로는 구간마다 다르게 성립한다는 점이다. 한 파이프라인 안에 유실이 나는 구간과 중복만 나는 구간이 동시에 있을 수 있고, " 이 시스템은 at-least-once 다" 라는 한 문장은 그 차이를 지워 버린다.

이 파이프라인은 producer(거래소 WS 를 구독해 Kafka 로 내보내는 프로세스) 3 종에서 시작해 Kafka, 그리고 ClickHouse 의 데이터 테이블까지 이어지는 사슬이다. 이 장은 그 사슬을 네 구간으로 쪼개, 각 구간이 실제로 어떤 보장을 갖는지 하나씩 확인한다.

*at-least-once(*적어도 한 번 전달*)* 란" 메시지가 유실되지는 않지만, 같은 메시지가 두 번 이상 도착할 수 있다" 는 보장이다. 반대로 *exactly-once(*정확히 한 번*)* 는" 유실도 없고 중복도 없다" 는, 더 강하지만 훨씬 지키기 어려운 보장이다. 스트리밍 시스템에서 진짜 exactly-once 를 배달 단계에서 보장하는 방법은 사실상 없다시피 하다. 대신 이 파이프라인이 택한 길은" 배달은 중복을 허용하되, 저장 단계에서 중복을 접어 *effectively-exactly-once(*사실상 정확히 한 번*)* 를 만든다" 이다. 절 5.5 에서 그 방법을 다룬다.

5.2 앱 내부 큐: at-least-once 조차 아닌 구간

무슨 문제가 있었나. producer 가 거래소 WS 로부터 메시지를 받으면, 그 메시지를 곧바로 Kafka 로 보내지 않는다. 먼저 애플리케이션 안의 큐 (`BufferedKafkaProducer.queue`) 에 쌓아 두고, `BATCH_SIZE`(100 개) 또는 `FLUSH_MS`(2000ms, 2 초) 조건 중 먼저 차는 쪽에서 한꺼번에 드레인 (비워서 내보냄) 한다.

이 큐는 순수 메모리 *(in-memory)* 다. 디스크에 쓰지 않고, 다른 프로세스로 복제하지도 않는다. 그래서 producer 프로세스가 죽으면, 그 순간 큐 안에 있던 메시지는 그대로 사라진다. 최악의 경우 `FLUSH_MS` 가 다 차기 직전, 즉 최대 2 초분의 메시지가 한 번에 사라질 수 있다.

왜 이것이" 조용한" 유실인가

여기서 진짜 문제는 유실이 일어난다는 사실 자체가 아니다. 그 유실이 아무 흔적도 남기지 않는다는 점이다. 두 가지가 겹쳐서 이런 결과를 만든다.

1. `dr_cb: false`. *delivery report* 콜백은 Kafka 클라이언트 라이브러리 (librdkafka) 가" 이 메시지가 결국 브로커에 쓰였는지, 아니면 끝내 실패했는지" 를 앱에 알려 주는 통로다. 이 콜백이 꺼져 있으면, produce 가 최종적으로 실패해도 애플리케이션은 그 사실을 알 방법이 없다.

2. WS 피드는 재생 *(replay)* 이 없다. 거래소가"5 초 전에 보낸 그 메시지를 다시 보내 달라" 는 요청을 받아 주지 않는다. 한 번 놓친 메시지는 그 순간의 호가·체결 정보를 영원히 잃는 것과 같다.

왜 이것이" 최악의 종류" 의 버그인가. 버그에는 두 종류가 있다. 하나는 터지는 순간 로그나 에러 메시지, 크래시로 자기 존재를 알리는 버그다. 다른 하나는 아무 신호도 남기지 않고 그냥 결과가 조용히 틀려지는 버그다. 이 큐 유실은 후자다. 프로세스는 정상 종료 로그를 남기고, 재시작 후에도 아무 에러 없이 다시 동작한다. 잃어버린 2 초분의 데이터는 로그 어디에도, 메트릭 어디에도 나타나지 않는다. 유실이 있었다는 사실을 알아채려면 ClickHouse 에 쌓인 시계열에서 시간 구멍을 직접 찾아내는 수밖에 없는데, BTC 1 종처럼 메시지가 촘촘한 스트림에서는 그 구멍조차 눈에 띄기 어렵다. 발견되지 않는 버그는 고쳐지지도 않는다. 이것이" 발생을 완전히 막을 수는 없어도, 최소한 발생했다는 사실만은 반드시 알 수 있어야 한다" 는 원칙이 스트리밍 파이프라인 설계에서 특히 중요한 이유다.

여기에 셧다운 경합 (race) 하나가 더 있다. 프로세스가 종료 신호를 받은 뒤에도 아직 처리 중이던 메시지가 큐에 넣으려 시도하면, 이미 닫힌 연결이라 그 메시지는 버려지고 `warn` 로그 한 줄만 남는다 (`enqueue after disconnect dropped`). 이 경우는 앞의 두 경우와 달리 로그가 남기는 하지만, error 가 아니라 warn 이라 운영 관점에서 놓치기 쉽다.

정리하면, producer 내부 큐부터 Kafka 브로커에 데이터가 실제로 쓰이기까지의 구간은 *at-least-once* 조차 성립하지 않는다. " 적어도 한 번은 전달된다" 는 말 자체가 이 구간에는 적용되지 않으며, 그 사실이 로그에도 남지 않는다.

5.3 브로커부터 ClickHouse 까지: at-least-once 가 성립하는 구간

무슨 문제가 있었나. 메시지가 큐를 벗어나 Kafka 클라이언트 내부 버퍼로 넘어간 다음부터는 사정이 다르다. 여기서부터는 네트워크 문제나 브로커 일시 장애로 인한 실패를 재시도 *(retry)* 로 극복할 수 있는 구간이기 때문이다. 문제는" 재시도가 있다" 는 사실만으로는 안심할 수 없다는 것이다. 재시도가 있어도 ack 수준이 약하면 여전히 유실이 날 수 있고, 재시도가 만드는 중복을 받아줄 준비가 없으면 중복 자체가 새로운 문제가 된다.

성립하는 이유. librdkafka(Kafka 클라이언트 라이브러리) 는 ack(승인) 를 받지 못한 produce 요청을 `message.timeout.ms` 까지 자동으로 재시도한다. 이 재시도는 같은 바이트를 다시 보내는 것이므로, 같은 `idempotency_key`(멱등성 키) 와 같은 `time_ms` 를 가진 메시지가 그대로 다시 전송된다. 즉 여기서 생기는 중복은 전부 완전히 동일한 행이다. 이 성질이 절 5.5 의 중복 제거를 가능하게 만드는 전제가 된다.

ClickHouse 의 `ENGINE=Kafka` 테이블도 같은 원리로 동작한다. Kafka 엔진은 메시지를 읽어 *materialized view(*구체화 뷰*, MV)* 에 삽입을 마친 후에만 그 오프셋 (offset, 읽은 위치) 을 커밋한다. 삽입과 커밋 사이에서 ClickHouse 가 죽으

면, 재시작 후 같은 오프셋부터 다시 읽어 같은 블록을 다시 삽입한다. 이 역시 완전히 동일한 행의 중복을 만든다.

단서 하나: ack 수준. 코드는 `acks` 옵션을 명시하지 않으므로 librdkafka 의 기본값인 `request.required.acks = -1`(all, 모든 복제본의 ack 를 기다림) 이 적용된다. 그런데 이 파이프라인의 Kafka 클러스터는 *replication factor(*복제 계수*)* 가 *1* 이다. 복제본이 하나뿐이면" 모든 복제본의 ack" 는 결국" 리더 브로커 한 대의 ack" 와 같은 뜻이 된다. 즉 설정값은 `acks=all` 이지만 실질 효과는 ***`acks=1`*** 이다. 그 한 대가 디스크에 fsync(강제로 디스크에 씀) 하기 전에 죽으면, 재시도로도 메꿀 수 없는 유실이 난다. 이 단서는 절 5.7 에서" 정직하게 어디까지가 남은 위험인가" 를 정리할 때 다시 등장한다.

정리하면, 브로커부터 *ClickHouse* 데이터 테이블까지는 *at-least-once* 가 성립한다. 재시도와 지연 커밋이 만드는 중복은 전부 바이트 단위로 동일한 재전송이라는 성질을 가지며, 이 성질을 절 5.5 이 이용한다.

5.4 파이프라인 전체 지도

앞 두 절의 내용을 한 장의 그림으로 모으면 그림 5.1 와 같다. 화살표 왼쪽의 괄호 번호는 이 장에서 다룬 구간 번호이고, 화살표 위 라벨이 그 구간의 실제 보장 수준이다.

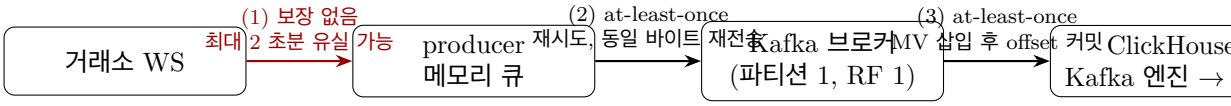


Figure 5.1: producer 큐부터 ClickHouse 데이터 테이블까지, 구간별 전달 보장 수준. (1) 만 보장이 없고 나머지는 at-least-once 이며, (4) 에서 effectively-exactly-once 로 접힌다.

5.5 DB 단에서 exactly-once 로 접는 방법

무슨 문제가 있었나. 절 5.3 에서 확인했듯, 브로커 이후 구간은 중복을 만들 수는 있어도 그 중복은 전부 바이트 단위로 동일한 재전송이다. 그렇다면 남은 문제는 하나다. 동일한 행이 여러 번 들어왔을 때, 저장 단계에서 그것을 자동으로 한 행으로 되돌릴 방법이 필요하다는 것이다.

그래서 무엇을 바꿨나. 데이터 테이블의 엔진으로 `ReplacingMergeTree(ingested_at)` 를 쓰고, 정렬 키 (sort key) 를 `ORDER BY (venue, ticker, time, idempotency_key)` 로 잡는다.

왜 그 변경이 문제를 푸는가. *ReplacingMergeTree* 는 백그라운드에서 파트 (part, 저장 단위) 를 병합 (merge) 할 때, 정렬 키가 완전히 같은 행들만 하나로 접는다. 여기서" 완전히 같다" 는 것은 `idempotency_key` 하나만 같아서는 안 되고, `venue`, `ticker`, `time` 까지 네 컬럼 전부가 같아야 한다는 뜻이다. 정렬 키의 일부만 같은 행은 별개의 행으로 남는다.

이 규칙이 이 파이프라인에서 실제로 작동하는 이유는, 절 5.3 에서 확인한 성질 때문이다. 이 파이프라인의 모든 중복은 바이트 단위로 동일한 재전송이므로, 같은 이벤트가 두 번 들어오면 `venue`, `ticker`, `time`, `idempotency_key` 네 값이 매번 똑같다. 여기서 `time` 은 ClickHouse 가 메시지를 받은 시각이 아니라, MV 가 페이로드의 `time_ms` 를 `toDateTime64(time_ms/1000, 3)` 으로 변환해 얻은 값이다. 즉 *payload* 안의 값에서 나온다. 재전송이 일어나도 페이로드 자체는 바뀌지 않으므로 `time` 도 매번 같고, 결과적으로 정렬 키 네 컬럼 전부가 자동으로 일치해 병합 시 접힌다.

이 불변식을 깨는 유일한 경우

정렬 키가 완전히 같아야 접힌다는 규칙은 거꾸로, 애플리케이션이 같은 이벤트를 다른 *time_ms* 로 다시 만들어 내면 중복 제거가 작동하지 않는다는 뜻이기도 하다. 예를 들어 producer 가 재시작한 뒤, 재시도로 보낸 것이 아니라 같은 오더북 상태를 새로운 *Date.now()* 로 다시 *publish* 하면, 그 두 행은 `time` 이 달라 정렬 키가 일치하지 않고 이력에 별개의 두 행으로 남는다. 이것은 재전송이 아니라 새로운 이벤트로 취급되는 것이므로, ReplacingMergeTree 입장에서는 정상 동작이다.

`venue` 가 정렬 키 선두에 있어야 하는 이유. Kalshi 의 티커는 `KXBTCD-26SEP22-B112000` 처럼 시리즈 형태 문자열이고, Polymarket 의 티커는 CLOB(Central Limit Order Book) token id 로, 수십 자리 10 진수 문자열이다. 두 거래소의 티커 값이 우연히 같은 문자열이 되는 일은 실무적으로 없다고 봐도 되지만, 그 가능성을 완전히 배제하려면 `venue` 가 정렬 키와 `idempotency_key` 양쪽 모두에 들어가야 한다.

`venue` 가 없다면, 두 거래소의 행이 `ticker` 만으로 구분되므로, 티커 문자열이 우연히 겹치는 극단적인 경우 서로 다른 거래소의 행이 하나로 잘못 접힐 수 있다. `venue` 를 정렬 키 선두에 두면 이 위험이 원천적으로 사라진다. 동시에 `venue` 는 카디널리티 (값의 종류 수) 가 2 4 개로 낮아, ClickHouse 의 스파스 인덱스 (sparse index) 가 `WHERE venue='kalshi'` 조건을 인덱스 단계에서부터 효율적으로 잘라낼 수 있다는 부수적인 이득도 있다.

예제 5.1 (venue 가 없었다면 벌어졌을 일) 가상의 상황을 생각해 보자. *Kalshi* 티커 *KXBTCD-26SEP22-B112000* 과 *Polymarket* 의 어떤 *CLOB token id* 가 *(*실제로는 형태가 전혀 달라 일어나지 않지만*)* 문자열로 우연히 같다고 가정하면, *venue* 없이 *ORDER BY (ticker, time, idempotency_key)* 만 쓸 때 두 거래소의 서로 다른 오더북 행이 같은 *time*, 같은 *idempotency_key* 를 우연히 공유하

는 순간 하나로 접혀 버린다. 서로 다른 시장의 데이터가 뒤섞이는 것이다. *venue* 를 선두에 추가하면 *(kalshi, KXBTCD-...)* 와 *(polymarket, KXBTCD-...)* 는 애초에 다른 정렬 키를 갖게 되어 이 위험이 사라진다.

5.6 왜 Kafka 트랜잭션 (EOS) 이 아니라 DB 중복 제거인가

무슨 문제가 있었나. Kafka 는 exactly-once semantics(EOS) 라는 자체 트랜잭션 기능을 제공한다. " 그러면 그걸 쓰면 되지 않느냐" 는 질문이 자연스럽게 나온다.

그래서 무엇을 바꿨나. 쓰지 않기로 했다. 결정적인 이유는 하나다. *ClickHouse* 의 *ENGINE=Kafka* 테이블은 트랜잭션 컨슈머 *(transactional consumer)* 가 아니다. Kafka EOS 는 트랜잭션을 지원하는 컨슈머 위에서만 의미가 있는데, 이 파이프라인의 읽기 쪽 (ClickHouse Kafka 엔진) 이 애초에 그 조건을 만족하지 못한다.

왜 그 변경이 문제를 푸는가. DB 단 중복 제거는 Kafka EOS 보다 적용 범위가 더 넓다. Kafka EOS 는 Kafka 내부의 read-process-write 패턴만 커버하지만, DB 단 중복 제거는 producer 크래시, 브로커 재시도, 컨슈머 재읽기까지 경로에 상관없이 전부 커버한다. 또한 producer 가 재시작해 PID(프로세스 ID) 가 바뀌어도 `idempotency_key` 가 안정적으로 재계산되는 한 중복 제거가 그대로 유지된다. Kafka EOS 는 PID 가 바뀌면 트랜잭션 보장이 사라진다.

	DB 중복 제거 (채택)	Kafka 트랜잭션 EOS
적용 범위	producer 크래시, 브로커 재시도, 컨슈머 재읽기 전부	Kafka 내부 read-process-write 만
ClickHouse Kafka 엔진	지원	트랜잭션 컨슈머 미지원. 애초에 쓸 수 없음
producer 재시작 후	`idempotency_key` 가 안정적이면 중복 제거 유지	PID 가 바뀌면 보장 소멸
비용	병합 시 CPU, 읽기 시 `FINAL`	처리량 저하, 운영 복잡도

Table 5.1: DB 단 중복 제거와 Kafka 트랜잭션 EOS 비교.

5.7 정직하게 짚는 잔존 위험

무슨 문제가 있었나. 지금까지의 논의를" 이 파이프라인은 exactly-once 다" 로 요약하면 과장이다. *effectively-exactly-once* 라는 표현을 쓴 이유는, 그 보장이 성립하지 않는 지점이 실제로 남아 있기 때문이다. 이 절은 그 지점을 숨기지 않고 나열한다.

(1) 중복이 아니라 유실로 바뀌는 지점: `kafka_skip_broken_messages = 1`. ClickHouse Kafka 엔진은 이 설정 때문에, 들어온 메시지가 proto 스키마와 맞지 않으면 그 행을 그냥 버린다. 이것은 지금까지 이야기한" 중복" 과 성질이 다르다. 중복은 나중에 접히기라도 하지만, 이 경우는 처음부터 유실이다. 스키마를 바꿀 때 (장 3 의 3 단계 계약) 가 이 파이프라인 전체에서 가장 위험한 순간인 이유가 여기에 있다. proto, codec, DDL 세 단계의 순서를 지키는 규율이 이 위험을 막는 유일한 수단이다.

(2) `acks=all` 이 사실상 `acks=1` 인 지점. 절 5.3 에서 다룬 것처럼, replication factor 가 1 이라 `acks=all` 의 실질 효과는 `acks=1` 이다. 재시도는 있지만, 그 재시도가 기대는 브로커가 단 한 대뿐이라는 사실은 바뀌지 않는다. 그 한 대의 디스크가 완전히 망가지면, 재시도로도 메꿀 수 없는 유실이 난다.

(3) 순서가 보장되지 않는 지점. Kafka 토픽은 전부 파티션이 1 개이므로 얼핏 순서가 유지될 것 같지만, 실제로는 그렇지 않다. producer 가 `enable.idempotence: false` 로 설정되어 있고, librdkafka 의 `max.in.flight.requests.per.connection`(연결당 동시 전송 가능한 요청 수) 기본값이 작지 않아, 재시도된 배치가 원래보다 늦게 도착할 수 있다. 파티션이 1 개라는 사실은 이 문제를 막아 주지 않는다.

ClickHouse 의 이력 테이블은 `time` 기준으로 정렬해서 읽으므로 순서 뒤집힘이 무해하다. 문제는 Redis 에 저장하는 현재 상태다. 오더북 라이터가 오래된 스냅샷을 나중에 받으면, 손대지 않을 경우 최신 상태를 오래된 값으로 덮어써 버린다. 그래서 Redis 쓰기에는 별도의 방어가 필요하다. `seq`(시퀀스 번호) 가 아니라 `time_ms` 를 비교하는 *last-write-wins(LWW,* 나중에 쓴 값이 이김*)* 가드를 Lua 스크립트로 구현해, 기존에 저장된 `time_ms` 보다 작거나 같은 스냅샷은 무시한다. `seq` 가 아니라 `time_ms` 를 쓰는 이유는 `seq` 가 재접속 시 리셋될 수 있는 값이기 때문이다. 이 가드는 `ob:{venue}:{ticker}` 단위로 걸리므로, 한 거래소의 순서 뒤집힘이 다른 거래소의 상태에 영향을 주지 않는다.

(4) 중복 제거는 최종적 (eventual) 이다. ReplacingMergeTree 의 병합은 백그라운드에서 비동기로 일어난다. 병합이 일어나기 전에는 같은 이벤트의 중복 행이 그대로 조회에 노출된다. 이 파이프라인은 BTC 1 종의 데이터량에서 `SELECT ... FROM t FINAL`(병합을 강제로 앞당겨 읽는 옵션) 로 이 문제를 단순하게 해결한다. 대시보드의" 현재 오더북" 은 애초에 ClickHouse 가 아니라 Redis 에서 읽으므로 이 지연의 영향을 받지 않는다.

5.8 한 줄로 보는 경로별 요약

표 5.2 는 이 장에서 다룬 네 구간을 한 표로 모은 것이다.

구간	보장 수준	근거
producer 큐 → Kafka 브로커	보장 없음. 최대 2 초분 조용히 유실 가능	메모리 큐, `dr_cb: false`, WS 는 재생 불가
Kafka 브로커 → ClickHouse Kafka 엔진	at-least-once	librdkafka 재시도, MV 삽입 후 offset 커밋
ClickHouse Kafka 엔진 → 데이터 테이블	중복은 발생하되 전부 바이트 동일	재시도/재읽기가 같은 payload 를 반복
데이터 테이블 병합	effectively-exactly-once (최종적)	`ReplacingMergeTree` + 정렬 키 완전 일치

Table 5.2: producer 큐부터 ClickHouse 데이터 테이블까지, 구간별 전달 보장 요약.

예제 5.2 (숫자로 보는 세 가지 시나리오) 같은 *Kalshi* 오더북 이벤트 하나 *(venue=kalshi, ticker=KXBTCD-26SEP22-B112000, time_ms=1758500000123, seq=4711)* 를 예로 세 가지 경우를 비교한다.

***(a)* 브로커 *ack* 유실 후 재시도, 그리고 삽입-커밋 사이 크래시.** 같은 바이트가 *producer* 에서 두 번, *ClickHouse* 에서 다시 두 번 재처리되어 데이터 테이블에 임시로 *4* 개의 행이 쌓일 수 있다. 네 행 모두 *(venue, ticker, time, idempotency_key)* 가 동일하므로, 병합 후에는 *1* 개 행만 남는다.

***(b) producer* 재시작 후 같은 오더북 상태를 새 시각으로 재발행.** 이것은 재시도가 아니라 새 이벤트 생성이다. 새 *time_ms* 를 얻으므로 정렬 키가 달라지고, 이력에는 *2* 개의 서로 다른 행으로 남는다. 이것이 절 *5.5* 에서 말한 *"* 불변식을 깨는 유일한 경우*"* 의 실제 사례다.

***(c)* 같은 티커 문자열을 가진 두 거래소.** *venue* 가 다르므로 *Redis* 키는 *ob|kalshi|KXBTCD-26SEP22-B112000|4711* 과 *ob|polymarket|KXBTCD-26SEP22-B112000|4711* 로 처음부터 갈라지고, *ClickHouse* 정렬 키도 *venue* 컬럼이 다르므로 두 거래소의 행이 병합에서 하나로 접히는 일은 없다.

정리. producer 메모리 큐부터 브로커까지는 at-least-once 조차 아니다. 크래시가 나면 최대 2 초분이 조용히 사라지고, `dr_cb: false` 때문에 그 유실은 로그에도 남지 않는다. 브로커부터 ClickHouse 데이터 테이블까지는 at-least-once 이며, 그 구간의 모든 중복은 바이트 단위로 동일한 재전송이므로 `idempotency_key` 를 정렬 키에 포함한 `ReplacingMergeTree` 가 effectively-exactly-once 로 접는다. 다만 순서는 보장되지 않으므로 (idempotence off), Redis 의 현재 상태는 `time_ms` 기준 last-write-wins 가드로 별도로 방어한다.

Chapter 6

예측과 트레이딩

이 장은 지금까지 쌓은 데이터 (오더북 이력, 체결, 틱) 를 실제 예측·트레이딩 로직에 연결하는 단계를 다룬다. MVP1 은 데이터 파이프라인의 형태만 증명하므로, 이 장의 본문은 그 파이프라인 위에 무엇을 얹을 것인가가 실제로 설계된 뒤에 채운다.

[**TODO**] 예측/트레이딩 설계가 나오면 작성. 현재 00.architecture.mvp1.md 는 이 영역을 명시적으로 범위 밖 (§1 " 제외" 목록의 arbitrage/fair-value/Kelly/주문집행) 으로 두고 있으므로, 이 장은 그 설계 문서가 나온 뒤 채운다.

Bibliography

[1] system-architect. 00. architecture — mvp1 (btc data pipeline, mock-first). 00.architecture.mvp1.md. project_market_predict, 2026-09-22.